Machine learning

Lecture4: K-Nearest Neighbors (KNN)

Prof: Aissam BERRAHOU
Aissam.berrahou@ensias.um5.ac.ma
Laboratoire Smart Systems Lab

ENSIAS

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

2020-2021

Objectifs

- Implémenter l'algorithme des k plus proches voisins.
- Calculer distances et similarités pour différents types de représentations des données.
- Définir la frontière de décision de l'algorithme du plus proches voisins.
- Expliquer pourquoi l'algorithme des k plus proches voisins est susceptible de ne pas marcher en haute dimension.

Plan

- Introduction
- Principe
- Implémentation avec python
- Points forts et faibles de KNN
- TP

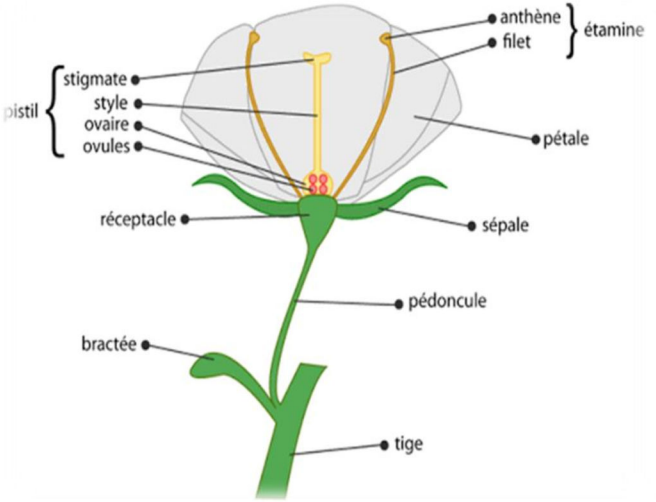
1. Introduction

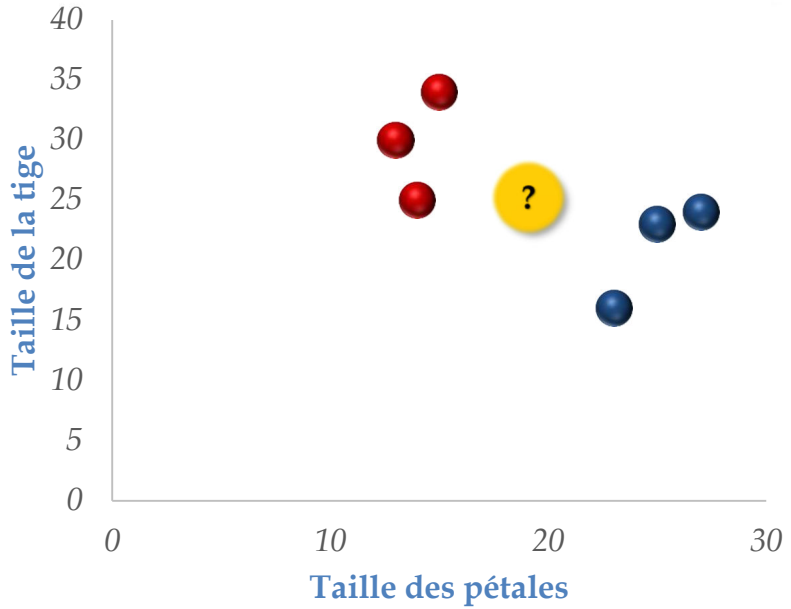
Introduction

- K Nearest Neighbors (KNN) en français **k-voisins les plus proches**.
  - Algorithme d'**apprentissage** automatique **supervisé**.
  - **Classification et régression.**
  - **Prédiction sur des variables qualitatives et quantitatives.**
  - Méthode **non paramétrique.**

Introduction

Taille des pétales	Taille de la tige	Catégorie
22	17	1
27	25	1
25	24	1
15	35	2
13	25	2
12	30	2
20	24	?





2. Principe

Principe

Algorithme: KNN( )

1 ▷ **Entrées:** ***E:*** *base apprentissage,* ***k:*** *nb de voisins plus proches,* ***x:*** *inconnu,* ***T:*** *Type_Class_Reg*

2 ▷ **Sortie:** ***R:*** *Résultat*

3 **Début**

4 **Pour chaque** donnée dans la base d'apprentissage;

5 Calculez la distance entre l'inconnu et la donnée courante;

6 Mémoriser cette distance et la donnée associée dans une liste de couple (distance,

7 donnée).

8 **Triez** la liste distance_Donnée du plus petit au plus grand sur les distances.

9 **Choisissez** les k premières entrées de cette liste.

10 **Obtenir** les étiquettes des k données de la base d'apprentissage sélectionnées.

11 **Si** T==Régression,

12 **renvoyer** (R=la moyenne des étiquettes K).

13 **Sinon** (c-à-d T==Classification)

14 **renvoyer** (R=l'étiquette majoritaire).

Principe

Classifier I(20,24)

Observations	Taille des pétales	Taille de la tige	Catégorie
O1	22	17	1
O2	27	25	1
O3	25	24	1
O4	15	35	2
O5	13	25	2
O6	12	30	2

Principe

Classifier I(20,24)

Observations	Taille des pétales	Taille de la tige	Catégorie	distance(I, O)
O1	22	17	1	7,280109889
O2	27	25	1	7,071067812
O3	25	24	1	5
O4	15	35	2	12,08304597
O5	13	25	2	7,071067812
O6	12	30	2	10

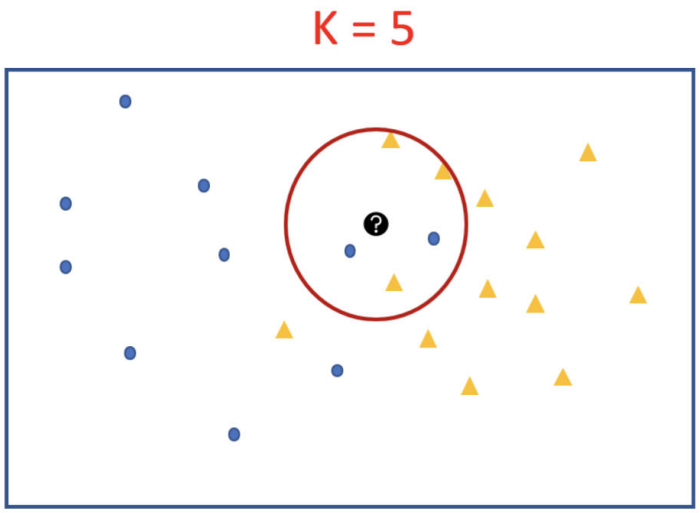
Principe

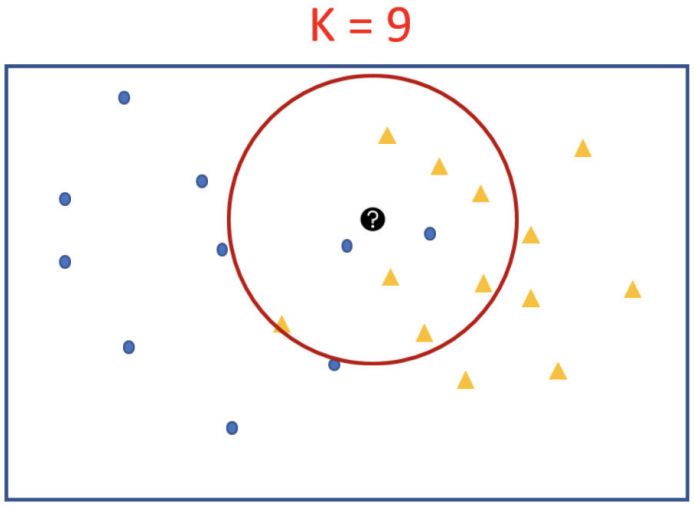
Classifier I(20,24)

Observations	Taille des pétales	Taille de la tige	Catégorie	distance(I, O)
O3	25	24	1	5
O2	27	25	1	7,071067812
O5	13	25	2	7,071067812
O1	22	17	1	7,280109889
O6	12	30	2	10
O4	15	35	2	12,08304597

Principe

- **Comment choisit-on ce paramètre k lors de l'implémentation de l'algorithme :**

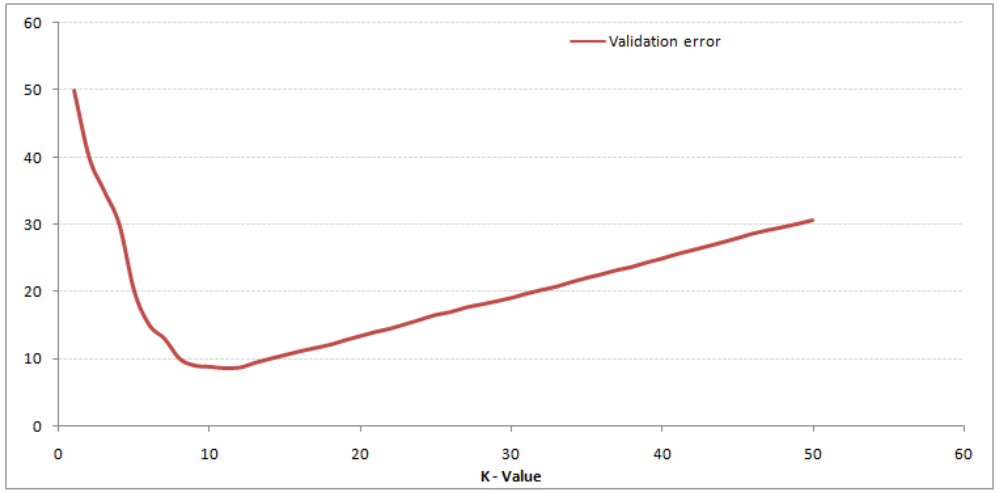




Principe

Comment choisit-on ce paramètre k lors de l'implémentation de l'algorithme :

- On fait varier k;
- Pour chaque valeur de k, on calcule le taux d'erreur de l'ensemble de test;
- On garde le paramètre k qui minimise ce taux d'erreur test.



Principe

Comment calculer la distance?

- **Distance euclidienne**:

$$d_e(U,V) = \sqrt{\sum_{i=1}^{n}(Ui - Vi)^2}$$

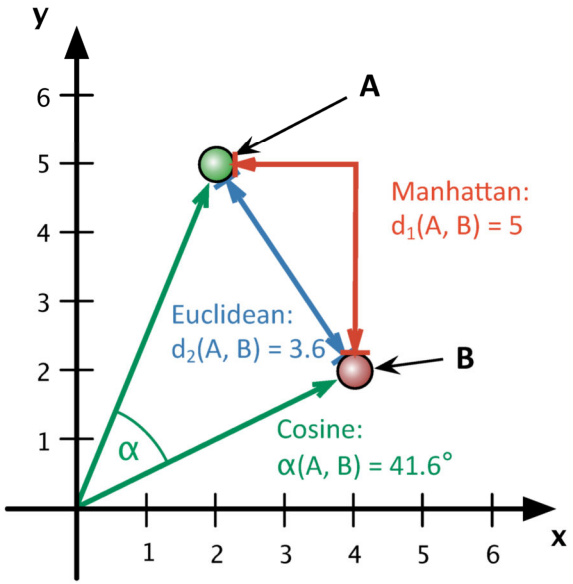
- Distance de manhattan:

$$d_m(U,V) = \sum_{i=1}^{n}|(Ui - Vi)|$$

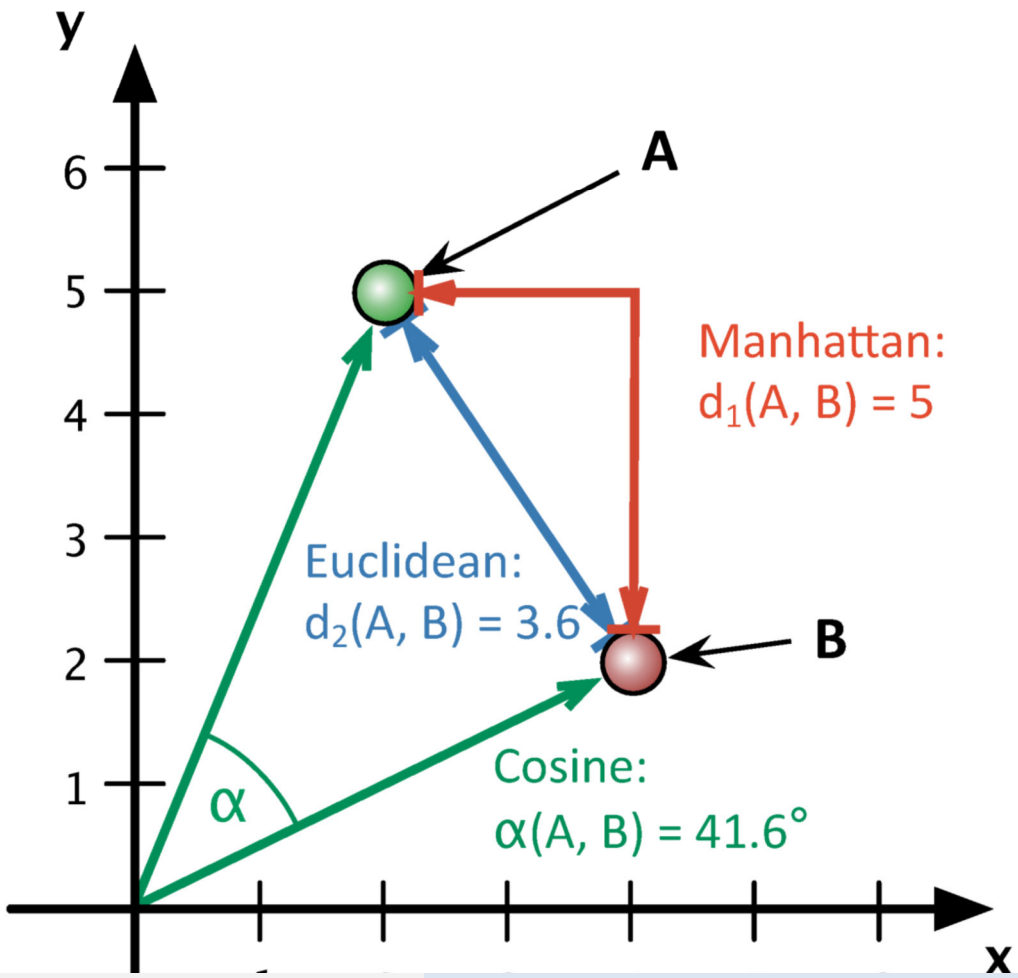
- Distance cosine :

$$d_c(U,V) = 1 - \frac{\sum_{i=1}^{n} U_i * Vi}{\sqrt{\sum_{i=1}^{n} U_i{}^2} * \sqrt{\sum_{i=1}^{n} V_i{}^2}}$$

$$= 1 - \frac{U * V^T}{\|U\| * \|V\|} = 1 - Cos(\alpha)$$

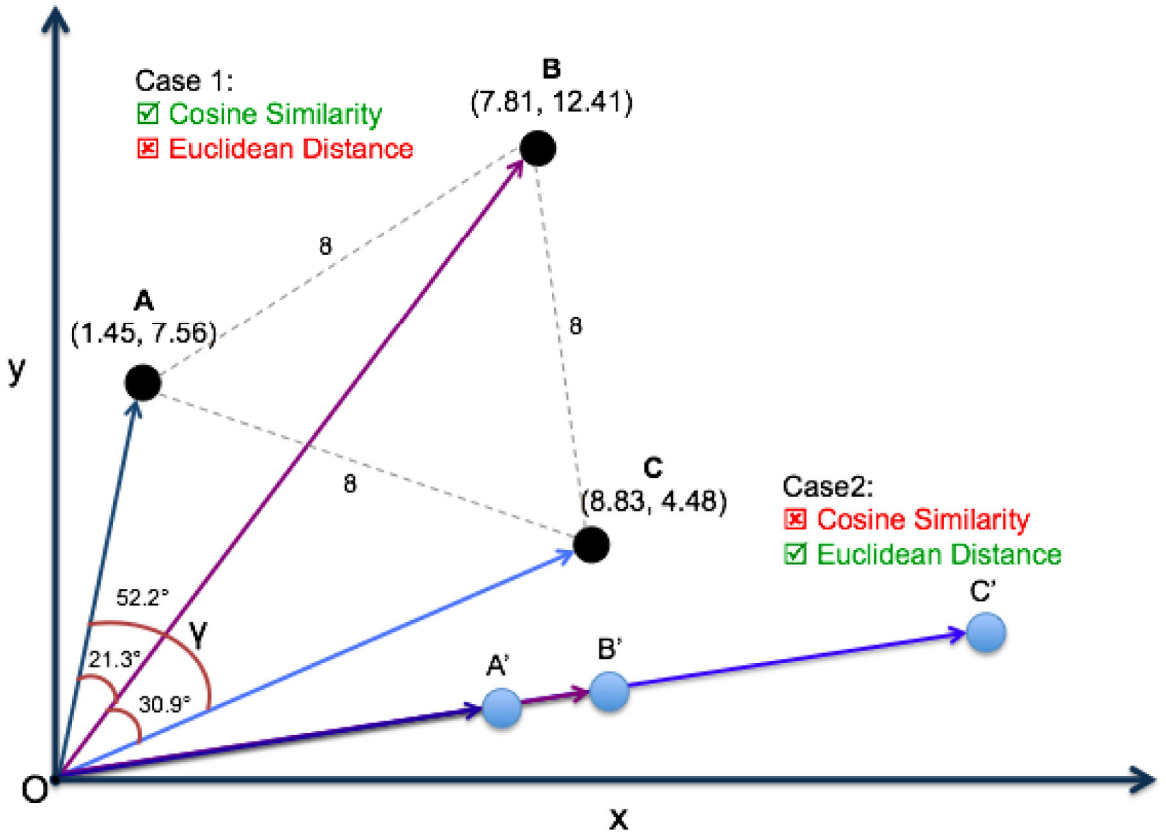


Principe


y
6
5
4
3
2
1
A
B
Manhattan:
$d_1(A, B) = 5$
Euclidean:
$d_2(A, B) = 3.6$
Cosine:
$\alpha(A, B) = 41.6°$
α
x

Principe

- Comment calculer la distance?



3. Implémentation

Implémentation avec python

- Exemple de classification de la base de données IRIS

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

# Chargement de la base de données IRIS
X = load_iris() .data
y = load_iris() .target
# Spliting de la BD
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size = 0.2, random_state=42)
#Implémentation de l'algorithme KNN

#Etudier le modèle en fonction du nombre de voisin et du type de distance
```

Implémentation avec sklearn

- Exemple de classification de la base de données IRIS

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
# Loading data
irisData = load_iris()

# Create feature and target arrays
X = irisData.data
y = irisData.target

# Split into training and test set
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size = 0.2, random_state=42)
knn = KNeighborsClassifier(n_neighbors=7) # k = 7
knn.fit(X_train, y_train)

# Calculate the accuracy of the model
print("Accuracy:", knn.score(X_test, y_test))
```

4. Points forts et faibles de KNN

Points forts et faibles de KNN

- Points forts:
  - \+ K-NN est un algorithme **assez simple à comprendre**;
  - \+ K-NN effectue des prédictions juste à temps (à la volée) en calculant la similarité entre un observation en entrée et les différentes observations de la Base de données;
- Points faibles:
  - \- **K-NN stocke toutes les exemples de la BD pour effectuer une prédiction**. Ainsi il faut faire attention à la **taille de l'ensemble d'apprentissage**.
  - \- KNN se base principalement sur le choix de la **méthode de calcul de la distance** ainsi que le **nombre de voisins** K **peut ne pas être évident**. Il faut essayer plusieurs combinaisons et faire du tuning de l'algorithme pour avoir un résultat satisfaisant.